

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE:CSA5115

SLOT:A

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Nissanki Sri Venkata Lakshmi Kavya(192373036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.S.V.L.Kavya

Annexure A

Experiments

1. **Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.**

PROCEDURE:

1. Client requests authentication.
2. Authentication Server (AS) generates a Ticket Granting Ticket (TGT).
3. Client uses the TGT to request a service ticket.
4. Server validates the ticket.
5. Timestamp/expiry prevents an old ticket from being reused in a replay attack.

PROGRAM:

```
import time
import hashlib
import hmac
class AuthenticationServer:
    def __init__(self):
        self.client_key = "client_secret"
        self.server_key = "server_secret"

    def encrypt(self, data, key):
        """Simple simulation of encryption using HMAC."""
        signature = hmac.new(
            key.encode(),
            data.encode(),
            hashlib.sha256
        ).hexdigest()

        return data + "|" + signature

    def verify(self, encrypted_data, key):
        try:
            data, signature = encrypted_data.rsplit("|", 1)

            expected = hmac.new(
                key.encode(),
                data.encode(),
                hashlib.sha256
            ).hexdigest()

            return hmac.compare_digest(signature, expected), data
```

```

except ValueError:
    return False, None

def generate_tgt(self, client_id):
    """Generate Ticket Granting Ticket."""
    timestamp = int(time.time())
    expiry = timestamp + 30

    ticket_data = f"{client_id}:{timestamp}:{expiry}"

    tgt = self.encrypt(ticket_data, self.client_key)

    print("\n[AS] TGT generated successfully.")
    return tgt

def generate_service_ticket(self, tgt, service):
    """Validate TGT and generate service ticket."""

    valid, data = self.verify(tgt, self.client_key)

    if not valid:
        print("[AS] Invalid TGT!")
        return None

    client_id, timestamp, expiry = data.split(":")

    if int(time.time()) > int(expiry):
        print("[AS] TGT expired!")
        return None

    service_ticket_data = (
        f"{client_id}:{service}:{int(time.time())}:"
        f"{int(time.time()) + 30}"
    )

    service_ticket = self.encrypt(
        service_ticket_data,
        self.server_key
    )

    print("[AS] Service ticket generated.")
    return service_ticket

class Server:
    def __init__(self):
        self.server_key = "server_secret"
        self.used_tickets = set()

```

```

def validate_ticket(self, ticket):
    valid, data = self.verify(ticket)
    if not valid:
        print("[Server] Invalid ticket!")
    return False
    client_id, service, timestamp, expiry = data.split(":")
    if int(time.time()) > int(expiry):
        print("[Server] Ticket expired!")
        return False
    if ticket in self.used_tickets:
        print("[Server] Replay attack detected!")
        return False
    self.used_tickets.add(ticket)

    print("[Server] Authentication successful!")
    print("[Server] Client:", client_id)
    print("[Server] Service:", service)
    return True
def verify(self, encrypted_data):
    try:
        data, signature = encrypted_data.rsplit("|", 1)
        expected = hmac.new(
            self.server_key.encode(),
            data.encode(),
            hashlib.sha256
        ).hexdigest()
        return hmac.compare_digest(signature, expected), data
    except ValueError:
        return False, None

client_id = "Client01"
service = "FileServer"
AS = AuthenticationServer()
server = Server()

print("==== KERBEROS AUTHENTICATION =====")
print("\n[Client] Requesting TGT...")
tgt = AS.generate_tgt(client_id)

print("[Client] Received TGT.")

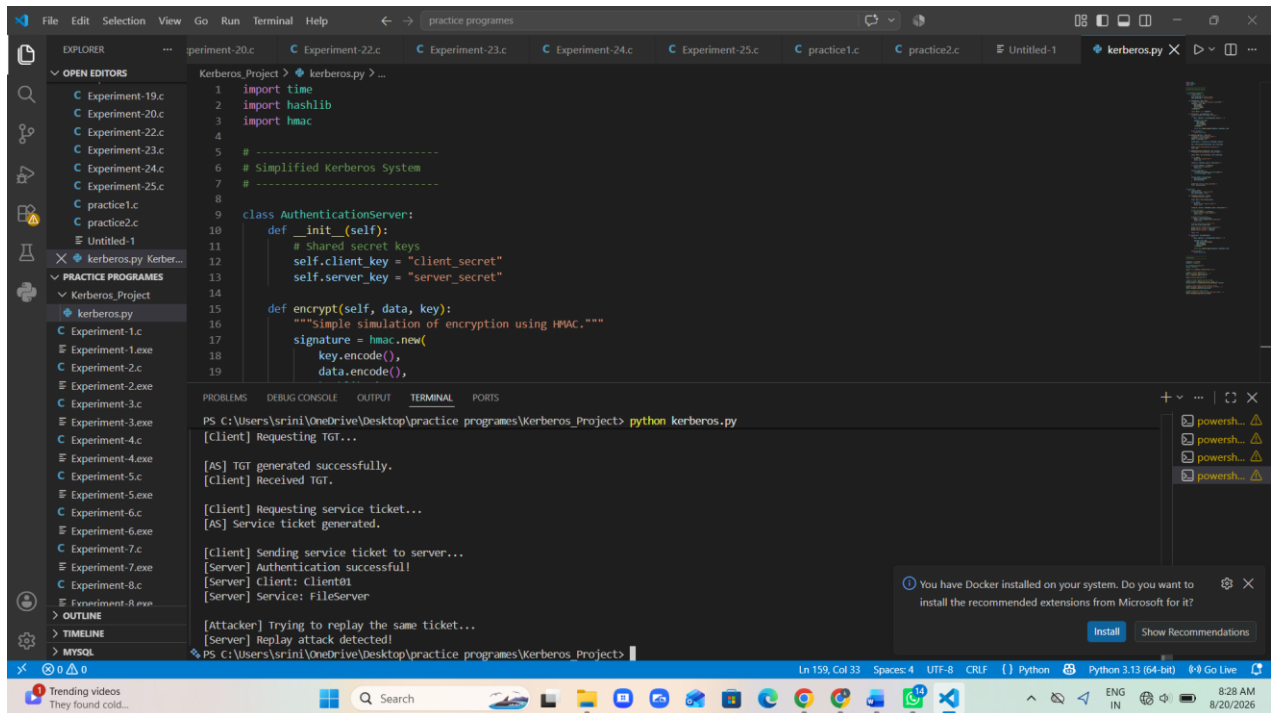
print("\n[Client] Requesting service ticket...")
service_ticket = AS.generate_service_ticket(tgt, service)

print("\n[Client] Sending service ticket to server...")
server.validate_ticket(service_ticket)

print("\n[Attacker] Trying to replay the same ticket...")
server.validate_ticket(service_ticket)

```

OUTPUT:



The screenshot shows a VS Code editor with a file explorer on the left containing various experiment and practice files. The main editor window displays a Python script named `kerberos.py` within a project called `Kerberos_Project`. The script implements a simplified Kerberos system with an `AuthenticationServer` class. The terminal at the bottom shows the output of running `python kerberos.py`, which simulates a successful authentication process between a client and a server, including TGT generation, service ticket request, and replay attack detection.

```
1 import time
2 import hashlib
3 import hmac
4
5 # -----
6 # Simplified Kerberos System
7 # -----
8
9 class AuthenticationServer:
10     def __init__(self):
11         # Shared secret keys
12         self.client_key = "client_secret"
13         self.server_key = "server_secret"
14
15     def encrypt(self, data, key):
16         """Simple simulation of encryption using HMAC."""
17         signature = hmac.new(
18             key.encode(),
19             data.encode(),
```

Terminal Output:

```
PS C:\Users\sriini\OneDrive\Desktop\practice programs\Kerberos_Project> python kerberos.py
[Client] Requesting TGT...
[AS] TGT generated successfully.
[Client] Received TGT.
[Client] Requesting service ticket...
[AS] Service ticket generated.
[Client] Sending service ticket to server...
[Server] Authentication successful!
[Server] Client: Client01
[Server] Service: FileServer
[Attacker] Trying to replay the same ticket...
[Server] Replay attack detected!
```

2. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

Aim

To develop a Python-based implementation of the Schnorr digital signature scheme and evaluate its security features compared with standard digital signature algorithms.

PROCEDURE:

1. Open VS Code and create `schnorr_signature.py`.
2. Import `hashlib` and `random`, and define the Schnorr parameters.
3. Generate the private key and corresponding public key.
4. Enter a message and generate its digital signature.
5. Verify the signature using the public key and test a modified message.
6. Run the program using `python schnorr_signature.py` and observe the results.

PROGRAM:

```
import hashlib
import random

class SchnorrSignature:

    def __init__(self):
        self.p = 23
        self.q = 11
        self.g = 2

        # Private key
```

```

self.private_key = random.randint(1, self.q - 1)

# Public key:  $y = g^x \bmod p$ 
self.public_key = pow(
    self.g,
    self.private_key,
    self.p
)

# Generate hash challenge
def hash_message(self, message, r):
    data = message + str(r)

    hash_value = hashlib.sha256(
        data.encode()
    ).hexdigest()

    return int(hash_value, 16) % self.q

# Sign a message
def sign(self, message):

    # Generate random nonce
    k = random.randint(1, self.q - 1)

    #  $r = g^k \bmod p$ 
    r = pow(self.g, k, self.p)

    #  $e = H(\text{message} \parallel r)$ 
    e = self.hash_message(message, r)

    #  $s = k + x \cdot e \bmod q$ 
    s = (k + self.private_key * e) % self.q

    return r, s

# Verify signature
def verify(self, message, signature):

    r, s = signature

    # Check valid range
    if r <= 0 or r >= self.p:
        return False

    if s < 0 or s >= self.q:
        return False

    # Calculate challenge
    e = self.hash_message(message, r)

```

```

        # Left side:  $g^s \bmod p$ 
        left = pow(self.g, s, self.p)

        # Right side:  $r * y^e \bmod p$ 
        right = (
            r * pow(self.public_key, e, self.p)
        ) % self.p

        return left == right

# -----
# Main Program
# -----

print("=====")
print("  SCHNORR DIGITAL SIGNATURE")
print("=====")

# Create Schnorr object
schnorr = SchnorrSignature()

print("\nPrivate Key:", schnorr.private_key)
print("Public Key :", schnorr.public_key)

# Message
message = input("\nEnter message: ")

print("\nOriginal Message:")
print(message)

# Generate signature
signature = schnorr.sign(message)

print("\nGenerated Signature:")
print("r =", signature[0])
print("s =", signature[1])

# Verify original message
result = schnorr.verify(message, signature)

print("\nSignature Verification:")
if result:
    print("Signature is VALID")
else:
    print("Signature is INVALID")
modified_message = message + " modified"

result2 = schnorr.verify(

```

```

        modified_message,
        signature
    )

print("\nModified Message Verification:")
if result2:
    print("Signature is VALID")
else:
    print("Signature is INVALID")

```

OUTPUT:

The screenshot shows a VS Code editor with a project named 'Kerberos_Project'. The file explorer on the left lists various experiment files and Python scripts. The main editor displays the file 'schnorr_signature.py' with the following code:

```

116
117 # Test modified message
118 modified_message = message + " modified"
119
120 result2 = schnorr.verify(
121     modified_message,
122     signature
123 )
124
125 print("\nModified Message Verification:")
126 if result2:
127     print("Signature is VALID")
128 else:
129     print("Signature is INVALID")

```

The terminal at the bottom shows the output of running 'python kerberos.py'. It displays a successful authentication sequence between a client and a server, followed by an attacker's attempt to replay the ticket, which is detected. The terminal also shows the output of running 'py schnorr_signature.py', which prints the digital signature and the private/public keys.

```

PS C:\Users\sriini\OneDrive\Desktop\practice programmes\Kerberos_Project> python kerberos.py

[Client] Sending service ticket to server...
[Server] Authentication successful!
[Server] Client: Client01
[Server] Service: FileServer

[Attacker] Trying to replay the same ticket...
[Server] Replay attack detected!
PS C:\Users\sriini\OneDrive\Desktop\practice programmes\Kerberos_Project> py schnorr_signature.py

=====
SCHNORR DIGITAL SIGNATURE
=====
Private Key: 1
Public Key : 2

```

A notification at the bottom right of the terminal area states: "You have Docker installed on your system. Do you want to install the recommended extensions from Microsoft for it?" with buttons for 'Install' and 'Show Recommendations'.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation